

Diseño y Desarrollo de Servicios Web



Tecnólogo en Análisis y Desarrollo de Software Ficha – 3118562

Instructor: Henry Alfonso Garzón Sánchez

Contenido

Introducción	3
Objetivo.....	4
Diseño de la Base de Datos.....	5
El Servidor	5
Configuración de la Conexión	6
Acceso a los Datos de Usuario.....	6
API de Autenticación.....	8
El Cliente	10
Centralizar las Rutas	10
Página de Inicio.....	11
Inicio de Sesión.....	12
Registro de Usuarios.....	15
Cierre de Sesión	16
Gestión de Estudiantes.....	17
Acceso a los Datos del Estudiante	18
API de Estudiantes.....	19
Formulario de Registro de Estudiantes	19
Listado de Estudiantes	20

Introducción

El presente documento corresponde a la evidencia del diseño y desarrollo de un servicio web construido con PHP nativo, el cual se encarga de gestionar la autenticación de usuarios y el registro de estudiantes. El proyecto aplica los conceptos de una API REST (Representational State Transfer - Application Programming Interface), un estilo de arquitectura utilizado en el desarrollo web.

El sistema se basa en la separación entre dos componentes que se comunican entre sí: un servidor, encargado de la lógica del sistema y el acceso a los datos, y un cliente, encargado de presentar las páginas con las que interactúa el usuario. Esta división permite que cada parte cumpla una responsabilidad específica; el cliente nunca consulta directamente la base de datos, sino que solicita la información al servidor mediante peticiones HTTP, y este responde en formato JSON (JavaScript Object Notation).

A lo largo de este documento se describe cada componente que conforma el sistema: el diseño de la base de datos, los archivos del servidor que gestionan la conexión, el acceso a los datos y las API, y los archivos del cliente que conforman las páginas de inicio de sesión, registro y gestión de estudiantes. Para cada componente se explica su funcionamiento general y los conceptos necesarios para comprender cómo opera el sistema en su conjunto.

Objetivo

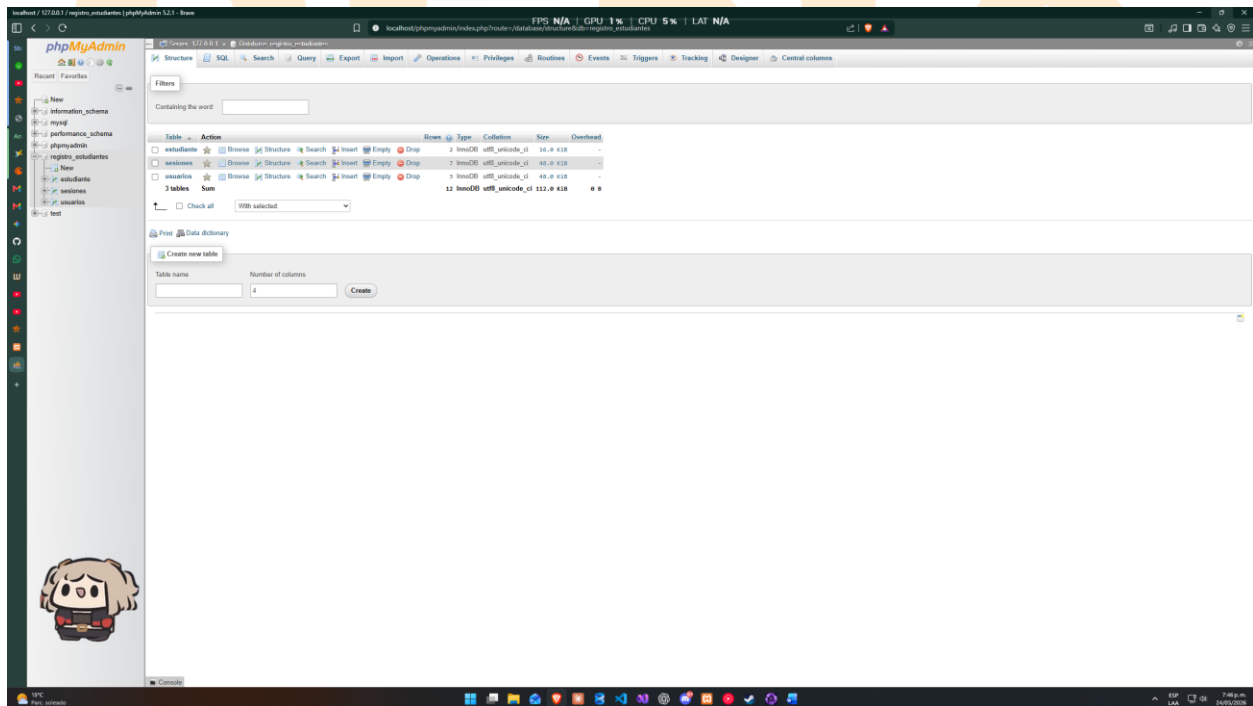
Desarrollar un servicio web en PHP nativo que implemente una API REST para la autenticación de usuarios y la gestión de estudiantes, aplicando el patrón de arquitectura cliente-servidor.



Diseño de la Base de Datos

Este sistema utiliza la base de datos de MySQL llamada **registro_estudiantes**. Para este proyecto se implementaron 3 tablas:

- Usuarios: almacena los usuarios que pueden iniciar sesión en el sistema.
- Sesiones. guarda los tokens de las sesiones que se encuentran activas.
- Estudiante. Almacena los estudiantes que se registran a través del formulario correspondiente.



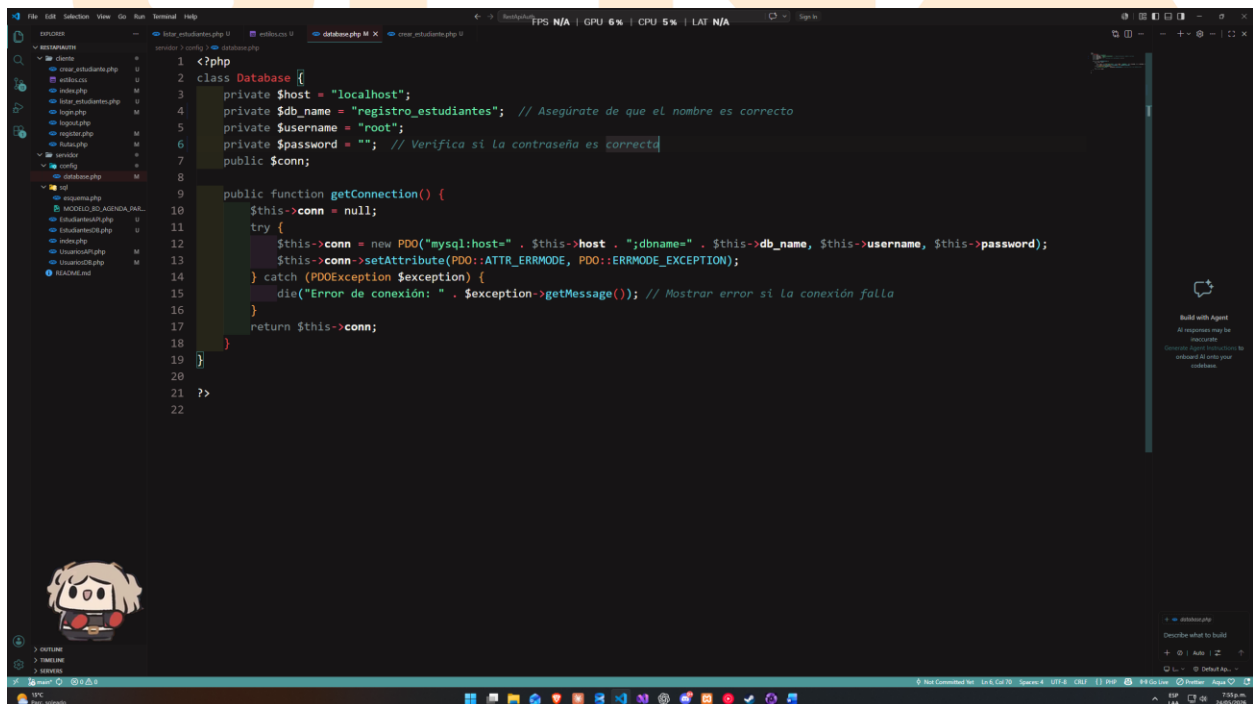
El Servidor

Es la parte del proyecto donde se encuentra la lógica y el acceso a los datos. Dentro de este se encuentran las funciones para recibir peticiones, procesarlas y devolver respuestas en formato JSON.

Configuración de la Conexión

En este archivo definimos la clase **Database**, que es la que se encarga de establecer la conexión con MySQL y contiene los datos de acceso (host, nombre de la base de datos, usuario y contraseña). También se define el método **getConnection()** que crea la conexión con la base de datos haciendo uso de **PDO (PDP Data Objects)** que es la forma en que PHP se conecta con bases de datos.

Este es el único punto dentro de todo el proyecto donde definimos las credenciales para la conexión con la **BD**, todos los demás archivos del servidor van a reutilizar esta clase en vez de repetirla una y otra vez para cada objeto que la necesite.

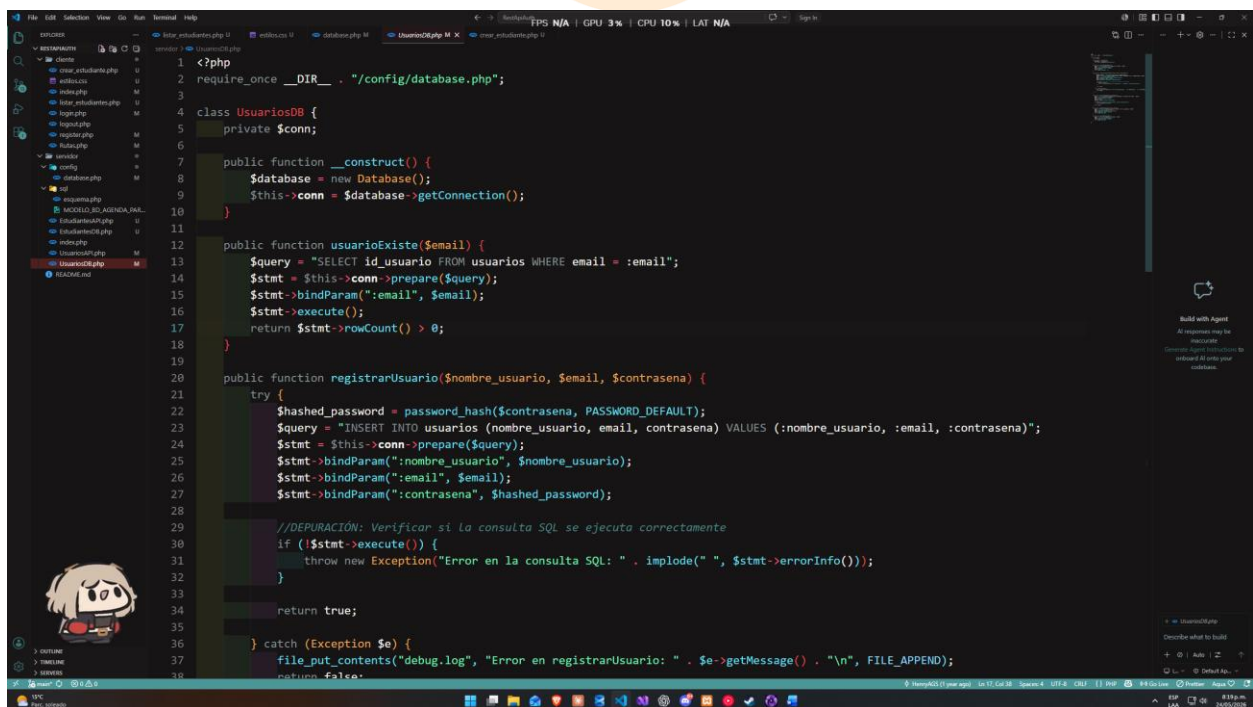


```
1 <?php
2 class Database {
3     private $host = "localhost";
4     private $db_name = "registro_estudiantes"; // Asegurate de que el nombre es correcto
5     private $username = "root";
6     private $password = ""; // Verifica si la contraseña es correcta
7     public $conn;
8
9     public function getConnection() {
10         $this->conn = null;
11         try {
12             $this->conn = new PDO("mysql:host=" . $this->host . ";dbname=" . $this->db_name, $this->username, $this->password);
13             $this->conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
14         } catch (PDOException $exception) {
15             die("Error de conexión: " . $exception->getMessage()); // Mostrar error si la conexión falla
16         }
17         return $this->conn;
18     }
19 }
20
21 ?>
22
```

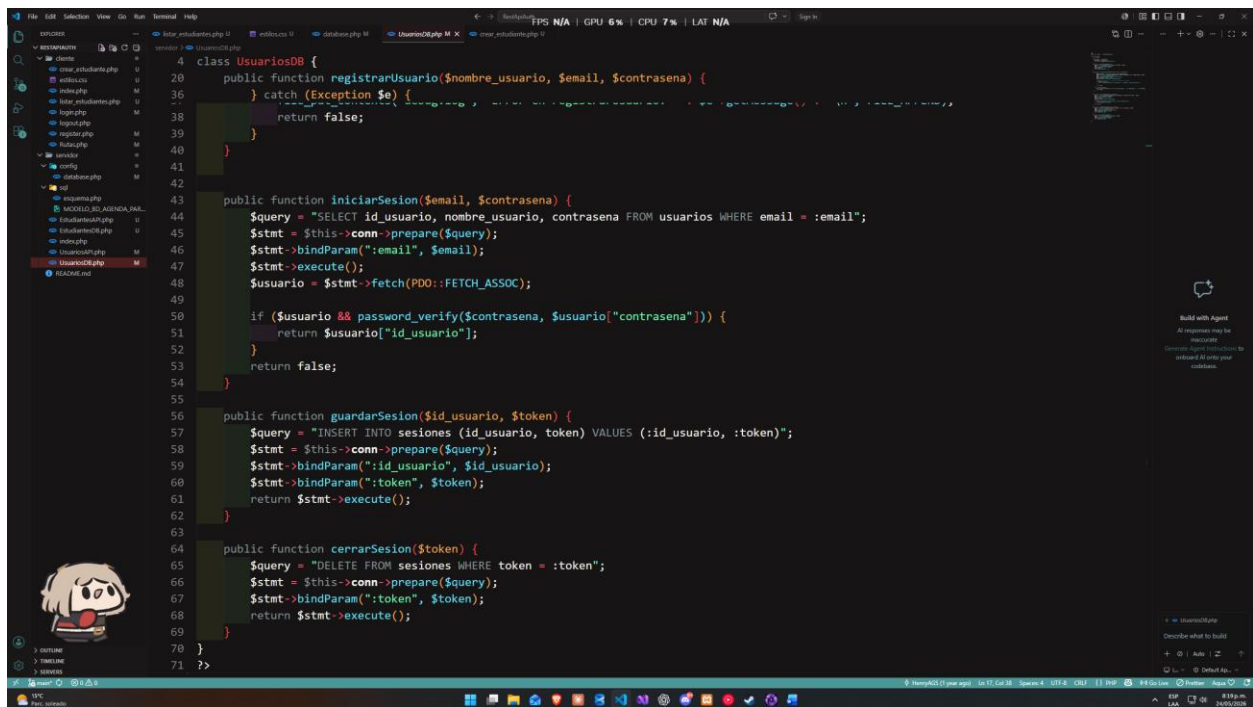
Acceso a los Datos de Usuario

Esta clase es la que se encarga de ejecutar las consultas SQL relacionadas con los usuarios y la que nos ayuda a gestionar los datos del módulo de autenticación. Sus tareas son las siguientes:

- Verificar si un usuario existe: con base a su correo electrónico.
- Registrar un usuario nuevo: antes de guardar la contraseña, la cifra con la función **password_hash()**.
- Inicio de sesión: Busca el usuario ingresado y lo compara con la contraseña ingresada con la almacenada haciendo uso de la función **password_verify()**.
- Guardar y cerrar sesión: administrado los tokens en la tabla de sesiones.



```
1 <?php
2 require_once __DIR__ . "/config/database.php";
3
4 class UsuariosDB {
5     private $conn;
6
7     public function __construct() {
8         $database = new Database();
9         $this->conn = $database->getConnection();
10    }
11
12    public function usuarioExiste($email) {
13        $query = "SELECT id_usuario FROM usuarios WHERE email = :email";
14        $stmt = $this->conn->prepare($query);
15        $stmt->bindParam(":email", $email);
16        $stmt->execute();
17        return $stmt->rowCount() > 0;
18    }
19
20    public function registrarUsuario($nombre_usuario, $email, $contrasena) {
21        try {
22            $hashed_password = password_hash($contrasena, PASSWORD_DEFAULT);
23            $query = "INSERT INTO usuarios (nombre_usuario, email, contrasena) VALUES (:nombre_usuario, :email, :contrasena)";
24            $stmt = $this->conn->prepare($query);
25            $stmt->bindParam(":nombre_usuario", $nombre_usuario);
26            $stmt->bindParam(":email", $email);
27            $stmt->bindParam(":contrasena", $hashed_password);
28
29            //DEPURACIÓN: Verificar si la consulta SQL se ejecuta correctamente
30            if (!$stmt->execute()) {
31                throw new Exception("Error en la consulta SQL: " . implode(" ", $stmt->errorInfo()));
32            }
33
34            return true;
35        } catch (Exception $e) {
36            file_put_contents("debug.log", "Error en registrarUsuario: " . $e->getMessage() . "\n", FILE_APPEND);
37            return false;
38        }
```

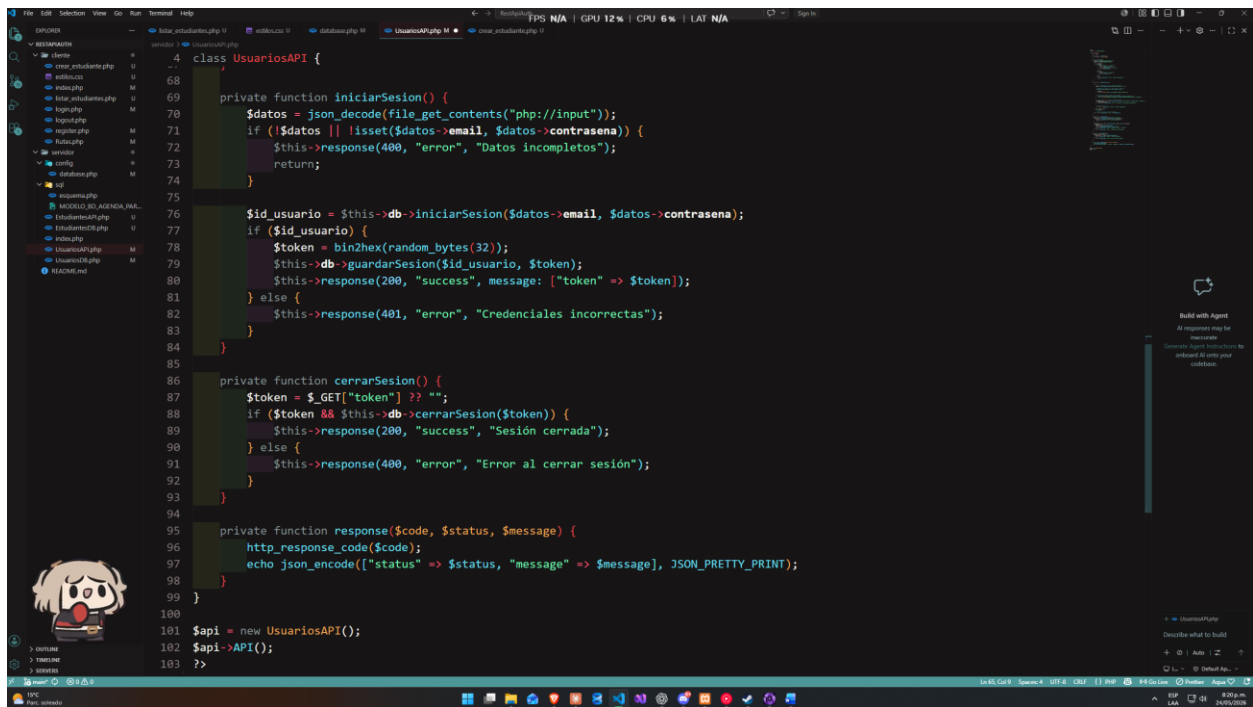


```
4 class UsuariosDB {
20 public function registrarUsuario($nombre_usuario, $email, $contrasena) {
36 } catch (Exception $e) {
38 return false;
39 }
40 }
41
42
43 public function iniciarSesion($email, $contrasena) {
44 $query = "SELECT id_usuario, nombre_usuario, contrasena FROM usuarios WHERE email = :email";
45 $stmt = $this->conn->prepare($query);
46 $stmt->bindParam(":email", $email);
47 $stmt->execute();
48 $usuario = $stmt->fetch(PDO::FETCH_ASSOC);
49
50 if ($usuario && password_verify($contrasena, $usuario["contrasena"])) {
51 return $usuario["id_usuario"];
52 }
53 return false;
54 }
55
56 public function guardarSesion($id_usuario, $token) {
57 $query = "INSERT INTO sesiones (id_usuario, token) VALUES (:id_usuario, :token)";
58 $stmt = $this->conn->prepare($query);
59 $stmt->bindParam(":id_usuario", $id_usuario);
60 $stmt->bindParam(":token", $token);
61 return $stmt->execute();
62 }
63
64 public function cerrarSesion($token) {
65 $query = "DELETE FROM sesiones WHERE token = :token";
66 $stmt = $this->conn->prepare($query);
67 $stmt->bindParam(":token", $token);
68 return $stmt->execute();
69 }
70 }
71 ?>
```

API de Autenticación

En este archivo esta la puerta para hacer uso de la API de autenticación. Existe el componente que recibe las peticiones que llegan desde el cliente. Sus funciones son las siguientes:

- Identifica la petición: revisa el método enviado por el navegador (HTTP) ya sea **POST** o **DELETE** más un parámetro llamado **action** para saber si el servicio que se esta solicitado es **registro, login o logout**.
- Procesar los datos: lee la información que es enviada en formato **JSON**, valida que no falten campos y llama a la clase **UsuariosDB**.
- Respuesta: una vez procesada la información el sistema devuelve un JSON con un **status** que puede decir “success o error” más un mensaje informativo para el usuario.



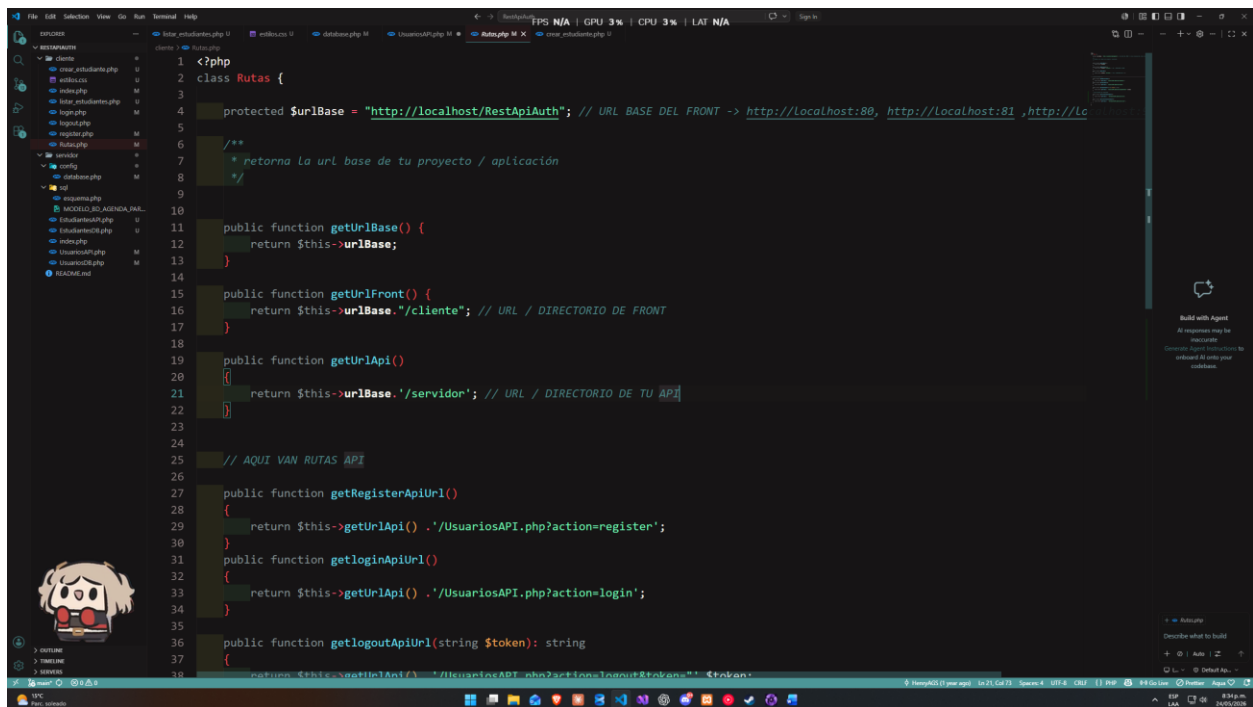
```
4 class UsuariosAPI {
68
69     private function iniciarSesion() {
70         $datos = json_decode(file_get_contents("php://input"));
71         if (!$datos || !isset($datos->email, $datos->contrasena)) {
72             $this->response(400, "error", "Datos incompletos");
73             return;
74         }
75
76         $id_usuario = $this->db->iniciarSesion($datos->email, $datos->contrasena);
77         if ($id_usuario) {
78             $token = bin2hex(random_bytes(32));
79             $this->db->guardarSesion($id_usuario, $token);
80             $this->response(200, "success", message: ["token" => $token]);
81         } else {
82             $this->response(401, "error", "Credenciales incorrectas");
83         }
84     }
85
86     private function cerrarSesion() {
87         $token = $_GET["token"] ?? "";
88         if ($token && $this->db->cerrarSesion($token)) {
89             $this->response(200, "success", "Sesión cerrada");
90         } else {
91             $this->response(400, "error", "Error al cerrar sesión");
92         }
93     }
94
95     private function response($code, $status, $message) {
96         http_response_code($code);
97         echo json_encode(["status" => $status, "message" => $message], JSON_PRETTY_PRINT);
98     }
99 }
100
101 $api = new UsuariosAPI();
102 $api->API();
103 ?>
```

El Cliente

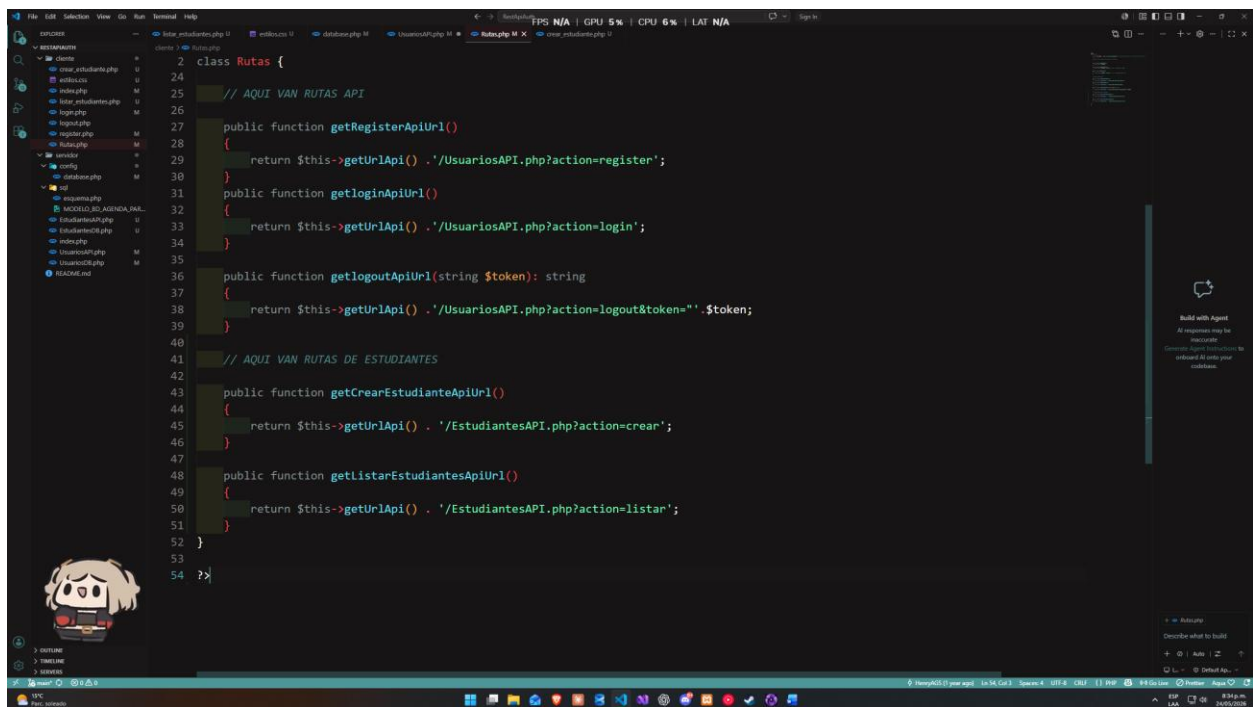
Esta es la parte del proyecto que es visible para el usuario y que además utiliza. Estas páginas no se comunican directamente con la base de datos, sino que se los piden al servidor mediante **cURL**.

Centralizar las Rutas

El archivo **Rutas.php** es una clase que reúne en un solo lugar todas las direcciones de los servicios de la API. Con ayuda de esto no tenemos que escribir la dirección de la API en cada página, sino que las mismas piden la URL a la clase correspondiente.



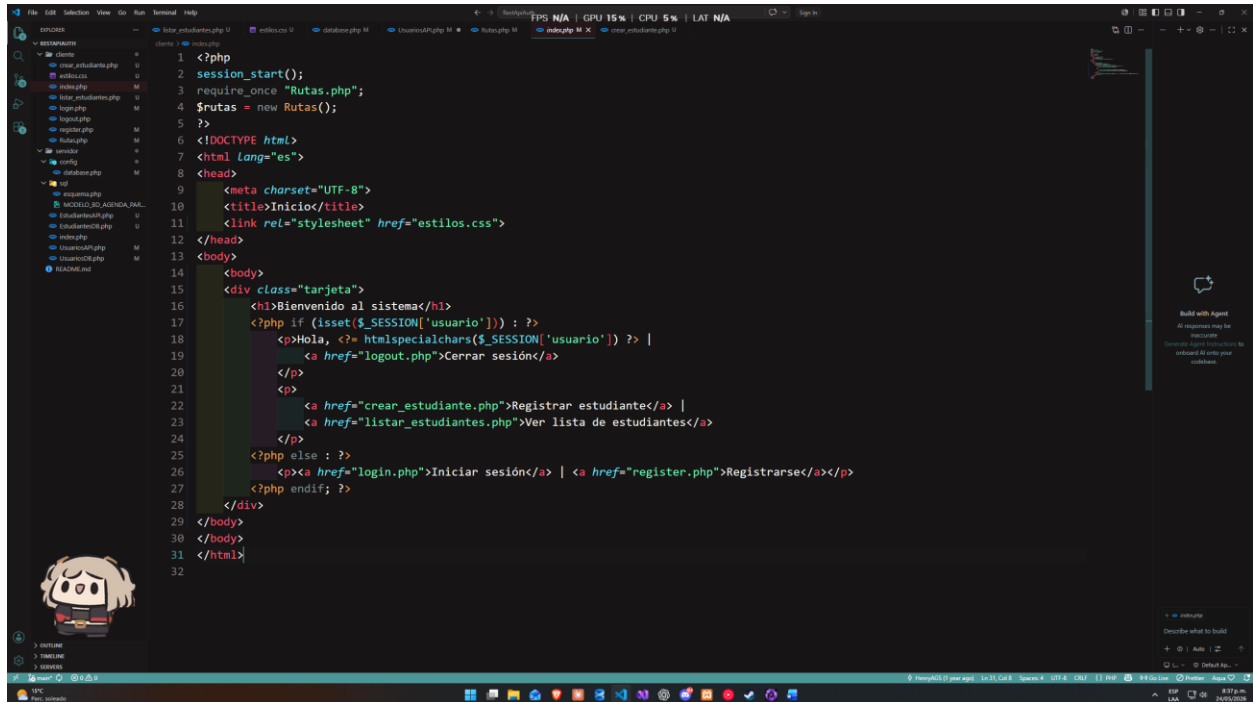
```
1 <?php
2 class Rutas {
3
4     protected $urlBase = "http://localhost/RestApiAuth"; // URL BASE DEL FRONT -> http://localhost:80, http://localhost:81, http://localhost:82
5
6     /**
7      * retorna la url base de tu proyecto / aplicación
8      */
9
10
11     public function getUrlBase() {
12         return $this->urlBase;
13     }
14
15     public function getUrlFront() {
16         return $this->urlBase."/cliente"; // URL / DIRECTORIO DE FRONT
17     }
18
19     public function getUrlApi() {
20         return $this->urlBase."/servidor"; // URL / DIRECTORIO DE TU API
21     }
22
23     // AQUI VAN RUTAS API
24
25     public function getRegisterApiUrl() {
26         return $this->getUrlApi() .'/UsuariosAPI.php?action=register';
27     }
28
29     public function getloginApiUrl() {
30         return $this->getUrlApi() .'/UsuariosAPI.php?action=login';
31     }
32
33     public function getlogoutApiUrl(string $token): string {
34         return $this->getUrlApi() .'/UsuariosAPI.php?action=logout&token='.$token;
35     }
36
37 }
```



```
24 class Rutas {
25
26     // AQUI VAN RUTAS API
27
28     public function getRegisterApiUrl() {
29         return $this->getUrlApi() .'/UsuariosAPI.php?action=register';
30     }
31
32     public function getloginApiUrl() {
33         return $this->getUrlApi() .'/UsuariosAPI.php?action=login';
34     }
35
36     public function getlogoutApiUrl(string $token): string {
37         return $this->getUrlApi() .'/UsuariosAPI.php?action=logout&token='.$token;
38     }
39
40     // AQUI VAN RUTAS DE ESTUDIANTES
41
42     public function getCrearEstudianteApiUrl() {
43         return $this->getUrlApi() .'/EstudiantesAPI.php?action=crear';
44     }
45
46     public function getListarEstudiantesApiUrl() {
47         return $this->getUrlApi() .'/EstudiantesAPI.php?action=listar';
48     }
49
50 }
51
52
53
54 ?>
```

Página de Inicio

Esta es la página principal que ve el cliente y la que comprueba si hay una sesión activa. Cuando el usuario inicia sesión el sistema da un saludo, y en caso contrario muestra los enlaces para iniciar sesión o registrarse.



```
1 <?php
2 session_start();
3 require_once "Rutas.php";
4 $rutas = new Rutas();
5 ?>
6 <!DOCTYPE html>
7 <html Lang="es">
8 <head>
9     <meta charset="UTF-8">
10    <title>Inicio</title>
11    <link rel="stylesheet" href="estilos.css">
12 </head>
13 <body>
14     <div class="tarjeta">
15         <h1>Bienvenido al sistema</h1>
16         <?php if (isset($_SESSION['usuario'])) : ?>
17             <p>Hola, <?= htmlspecialchars($_SESSION['usuario']) ?> |
18             <a href="logout.php">Cerrar sesión</a>
19         </p>
20         <p>
21             <a href="crear_estudiante.php">Registrar estudiante</a> |
22             <a href="listar_estudiantes.php">Ver lista de estudiantes</a>
23         </p>
24         <?php else : ?>
25             <p><a href="login.php">Iniciar sesión</a> | <a href="register.php">Registrarse</a></p>
26         <?php endif; ?>
27     </div>
28 </body>
29 </body>
30 </html>
```

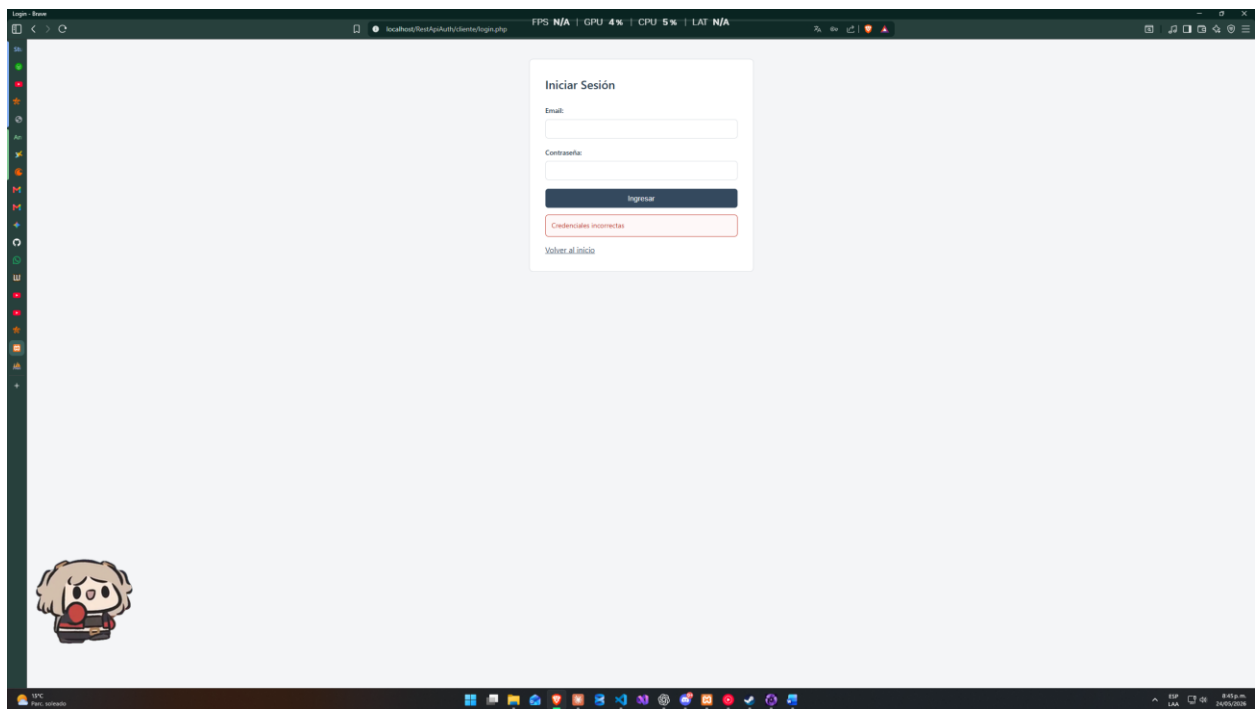
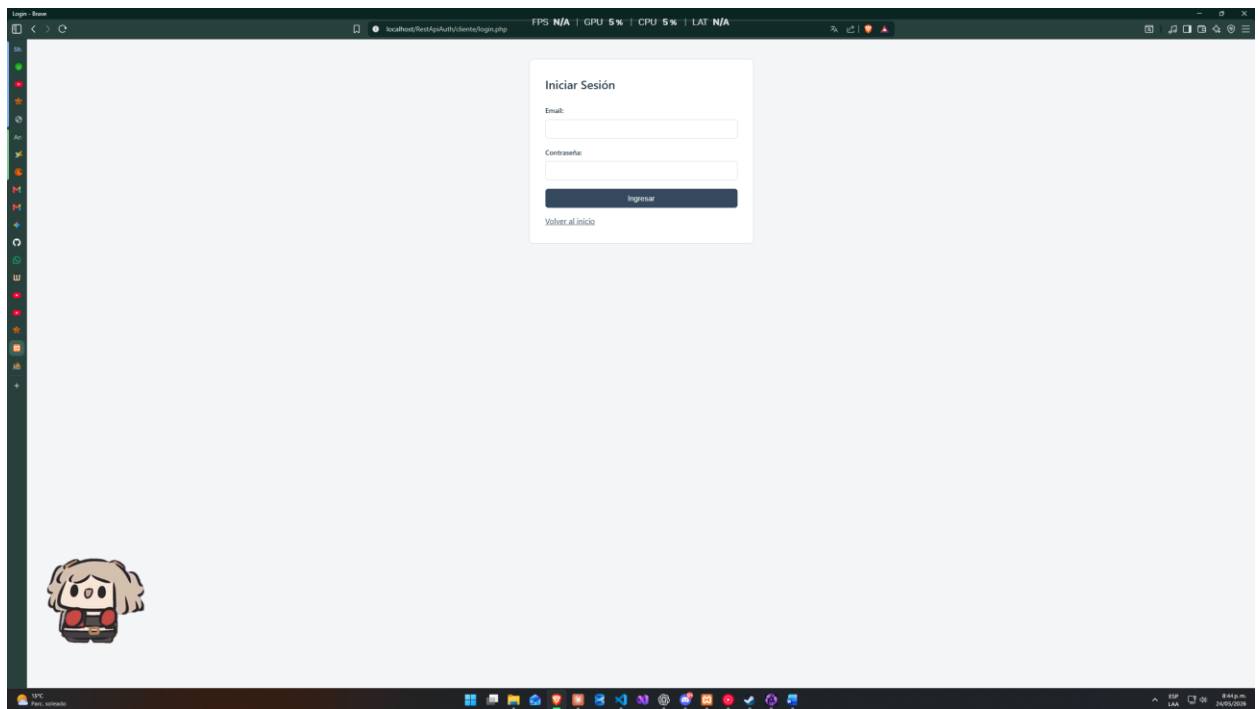
Inicio de Sesión

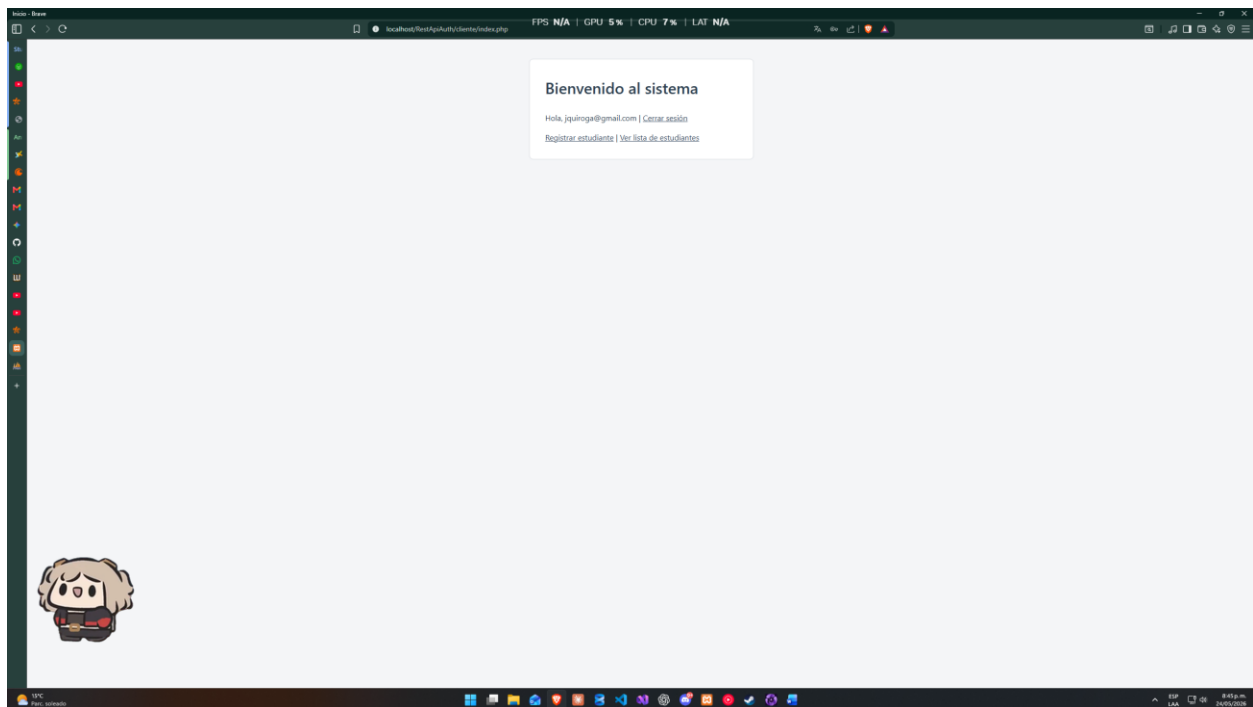
Este apartado está dividido en dos partes. La primera es la parte de PHP que se ejecuta únicamente cuando el usuario diligencia el formulario, el sistema toma el correo y la contraseña, los guarda en formato JSON y los envía para procesar por la API. Si la API responde correctamente el sistema guarda la sesión del usuario y lo redirige a la página de inicio, de lo contrario se muestra un mensaje de error.

La segunda parte es solamente el HTML con el formulario que ve el usuario.

```
1 <?php
2 session_start();
3 require_once('Rutas.php');
4
5 if ($_SERVER["REQUEST_METHOD"] == "POST") {
6     $rutas = new Rutas();
7
8     $email = filter_var($_POST["email"], FILTER_SANITIZE_EMAIL);
9     $contrasena = $_POST["contrasena"];
10    $data = json_encode(["email" => $email, "contrasena" => $contrasena]);
11
12    $ch = curl_init($rutas->getloginApiUrl());
13    curl_setopt($ch, CURLOPT_POST, 1);
14    curl_setopt($ch, CURLOPT_POSTFIELDS, $data);
15    curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
16    curl_setopt($ch, CURLOPT_HTTPHEADER, ["Content-Type: application/json"]);
17
18    $res = curl_exec($ch);
19    curl_close($ch);
20
21    $response = json_decode($res, true);
22
23    if ($response && isset($response['message'])['token']) {
24        $_SESSION["usuario"] = $email;
25        $_SESSION["token"] = $response['message']['token'];
26        header("Location: index.php");
27        exit;
28    } else {
29        $error = "Credenciales incorrectas";
30    }
31 }
32 ?>
33 <!DOCTYPE html>
34 <html Lang="es">
35 <head>
36     <meta charset="UTF-8">
37     <title>Login</title>
38     <link rel="stylesheet" href="estilos.css">
39 </head>
40 <body>
41     <div class="tarjeta">
42         <h2>Iniciar Sesión</h2>
43         <form method="POST">
44             <label>Email:</label>
45             <input type="email" name="email" required>
46             <label>Contraseña:</label>
47             <input type="password" name="contrasena" required>
48             <button type="submit">Ingresar</button>
49         </form>
50         <?php if (isset($error)) : ?>
51             <p class="mensaje mensaje-error"><?= $error ?></p>
52         <?php endif; ?>
53         <p><a href="index.php">Volver al inicio</a></p>
54     </div>
55 </body>
56 </body>
57 </html>
58 </html>
59
```

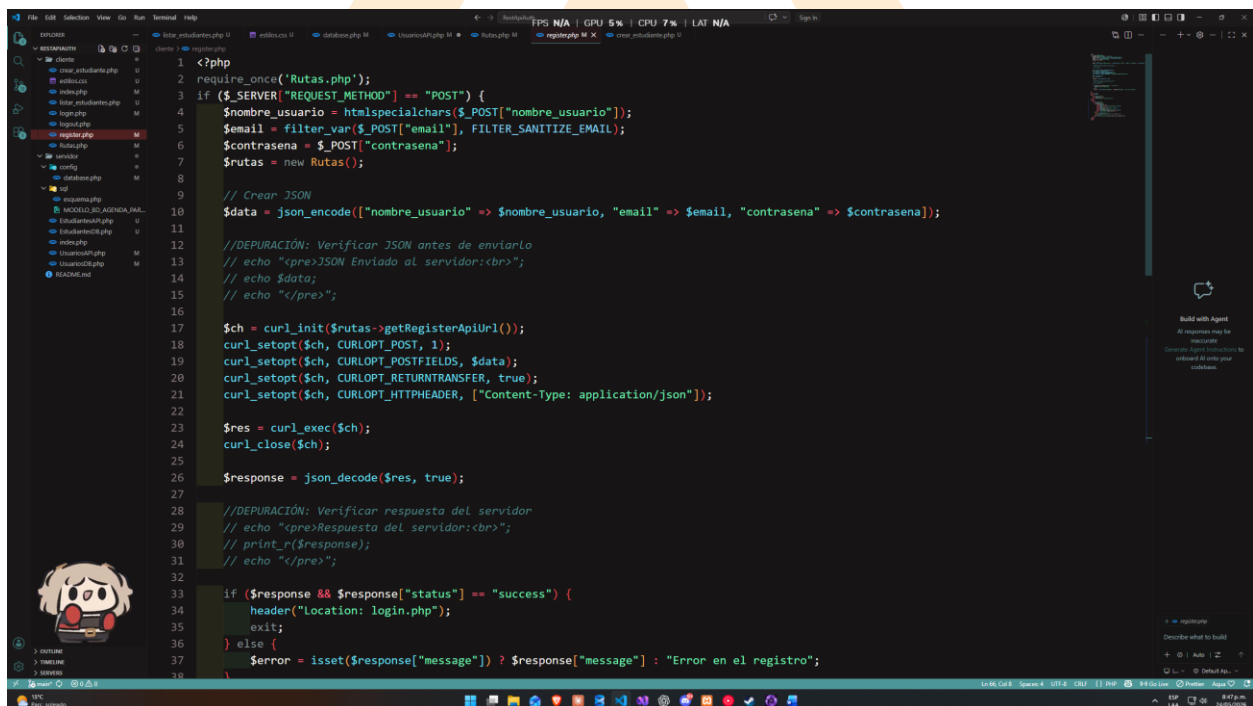
```
1 <?php
2 session_start();
3 require_once('Rutas.php');
4
5 if ($_SERVER["REQUEST_METHOD"] == "POST") {
6     $rutas = new Rutas();
7
8     $email = filter_var($_POST["email"], FILTER_SANITIZE_EMAIL);
9     $contrasena = $_POST["contrasena"];
10    $data = json_encode(["email" => $email, "contrasena" => $contrasena]);
11
12    $ch = curl_init($rutas->getloginApiUrl());
13    curl_setopt($ch, CURLOPT_POST, 1);
14    curl_setopt($ch, CURLOPT_POSTFIELDS, $data);
15    curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
16    curl_setopt($ch, CURLOPT_HTTPHEADER, ["Content-Type: application/json"]);
17
18    $res = curl_exec($ch);
19    curl_close($ch);
20
21    $response = json_decode($res, true);
22
23    if ($response && isset($response['message'])['token']) {
24        $_SESSION["usuario"] = $email;
25        $_SESSION["token"] = $response['message']['token'];
26        header("Location: index.php");
27        exit;
28    } else {
29        $error = "Credenciales incorrectas";
30    }
31 }
32 ?>
33 <!DOCTYPE html>
34 <html Lang="es">
35 <head>
36     <meta charset="UTF-8">
37     <title>Login</title>
38     <link rel="stylesheet" href="estilos.css">
39 </head>
40 <body>
41     <div class="tarjeta">
42         <h2>Iniciar Sesión</h2>
43         <form method="POST">
44             <label>Email:</label>
45             <input type="email" name="email" required>
46             <label>Contraseña:</label>
47             <input type="password" name="contrasena" required>
48             <button type="submit">Ingresar</button>
49         </form>
50         <?php if (isset($error)) : ?>
51             <p class="mensaje mensaje-error"><?= $error ?></p>
52         <?php endif; ?>
53         <p><a href="index.php">Volver al inicio</a></p>
54     </div>
55 </body>
56 </body>
57 </html>
58 </html>
59
```

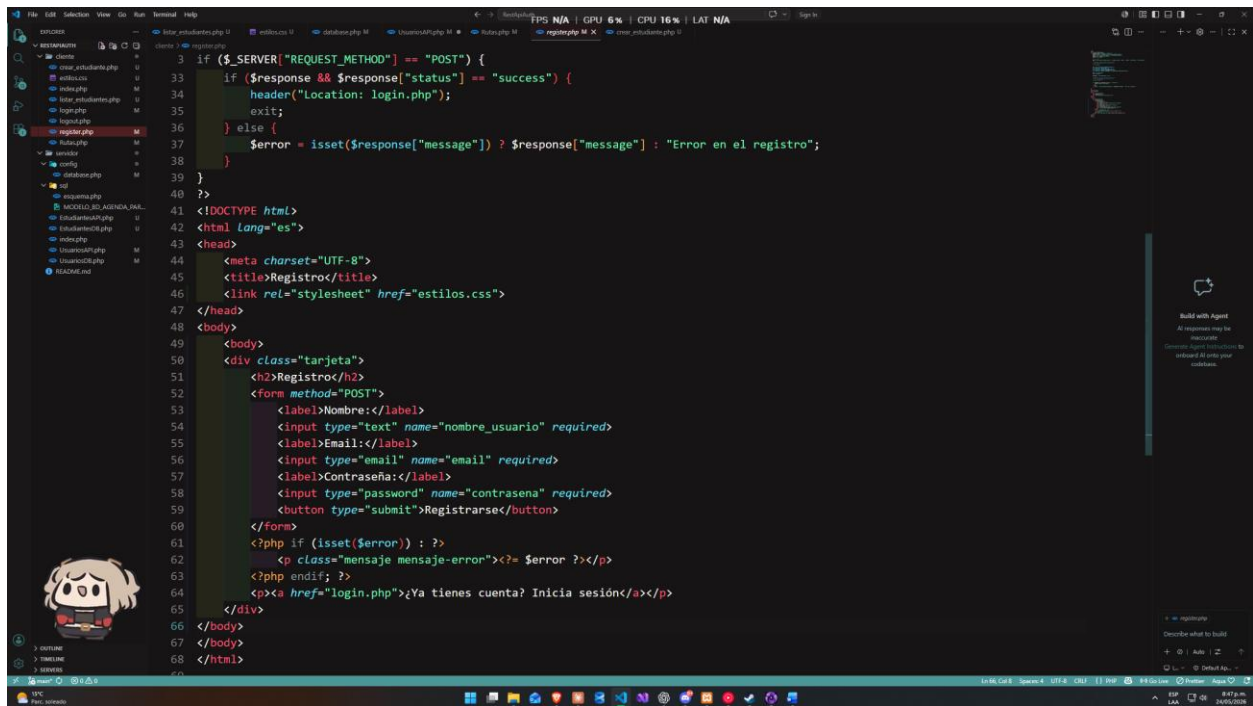




Registro de Usuarios

Su objetivo es únicamente el de crear un nuevo usuario, enviando los datos del formulario a la API con la acción de registro, el cual si es exitoso redirige a la página de inicio de sesión.

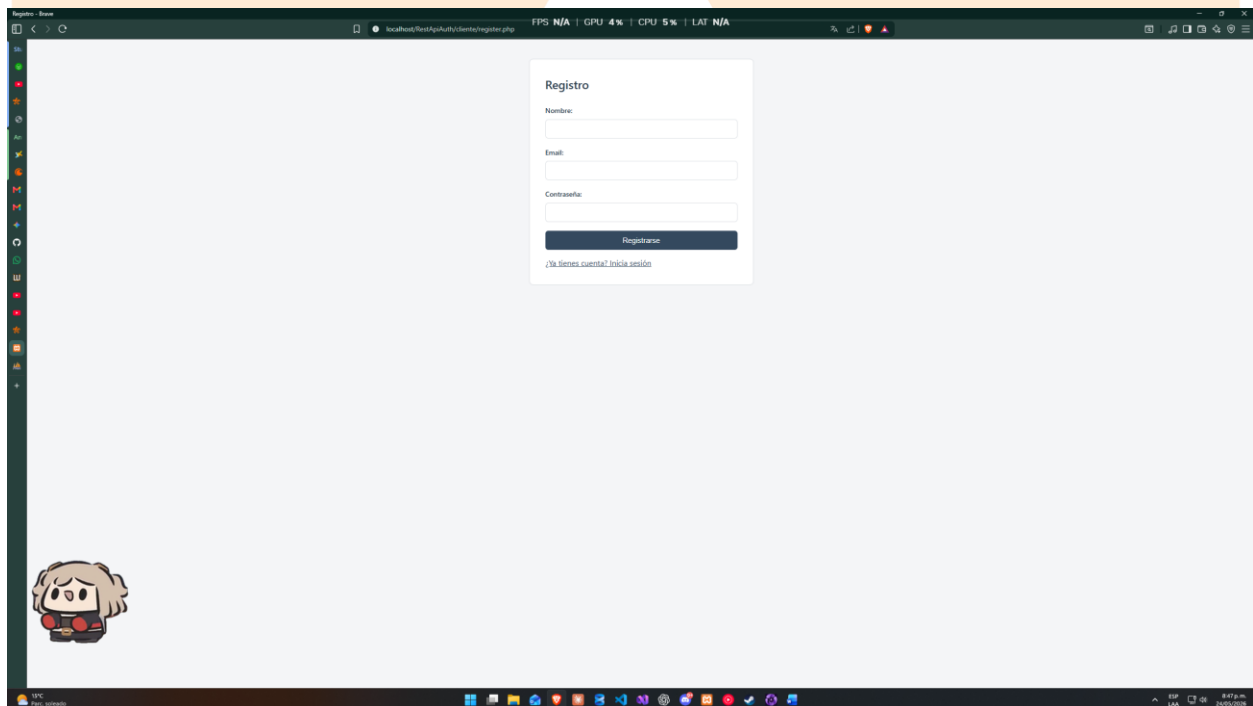




The screenshot shows a code editor with a dark theme. The left sidebar displays a file explorer with a project structure including folders like 'cliente' and 'server', and files like 'registro.php', 'login.php', 'logout.php', 'index.php', 'error.php', 'styles.css', 'README.md', 'MODELO_AGENCIA_PASA...', 'Estadisticas.php', 'Estadisticas.php', 'index.php', 'Usuarios.php', and 'Usuarios.php'. The main editor area shows the following code:

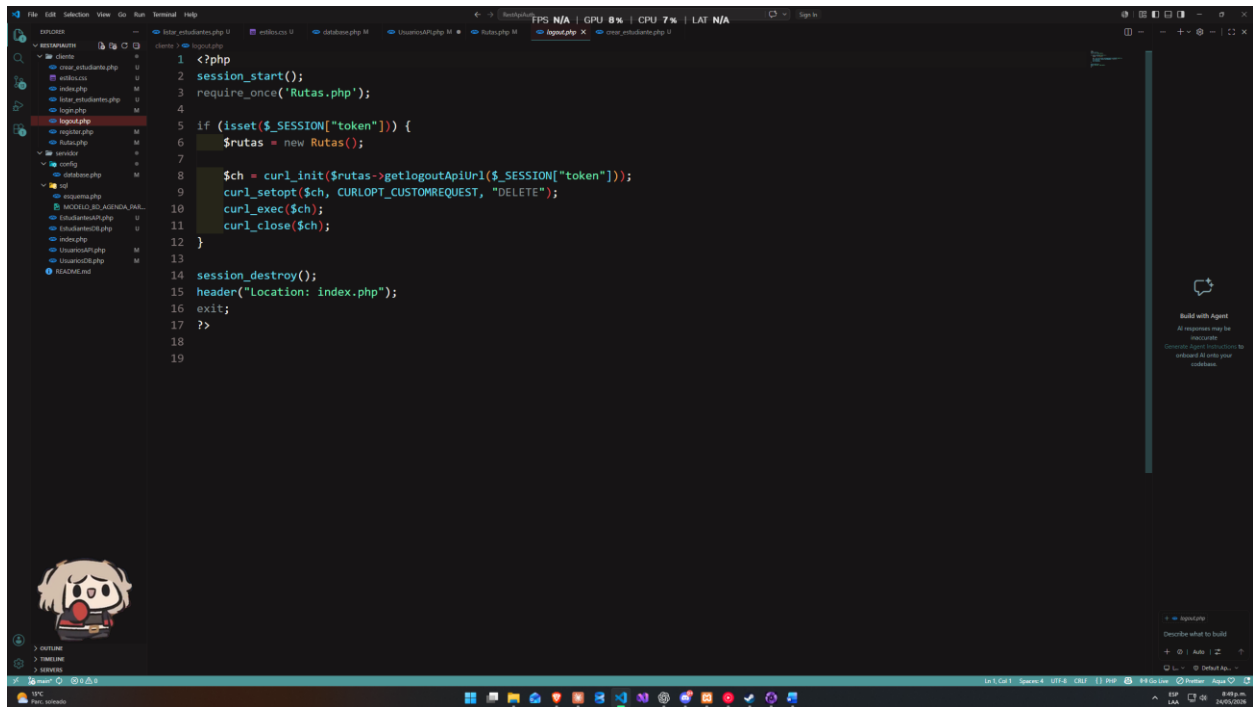
```
3 if ($_SERVER["REQUEST_METHOD"] == "POST") {
33     if ($response && $response["status"] == "success") {
34         header("Location: login.php");
35         exit;
36     } else {
37         $error = isset($response["message"]) ? $response["message"] : "Error en el registro";
38     }
39 }
40 }
41 <!DOCTYPE html>
42 <html Lang="es">
43 <head>
44     <meta charset="UTF-8">
45     <title>Registro</title>
46     <link rel="stylesheet" href="estilos.css">
47 </head>
48 <body>
49 <div class="tarjeta">
50     <h2>Registro</h2>
51     <form method="POST">
52         <label>Nombre:</label>
53         <input type="text" name="nombre_usuario" required>
54         <label>Email:</label>
55         <input type="email" name="email" required>
56         <label>Contraseña:</label>
57         <input type="password" name="contrasena" required>
58         <button type="submit">Registrarse</button>
59     </form>
60     <?php if (isset($error)) : ?>
61     <p class="mensaje mensaje-error"><?=$error ?></p>
62     <?php endif; ?>
63     <p><a href="login.php">¿Ya tienes cuenta? Inicia sesión</a></p>
64 </div>
65 </body>
66 </body>
67 </html>
```

The bottom status bar shows 'FPS N/A | GPU 6% | CPU 16% | LAT N/A'.



Cierre de Sesión

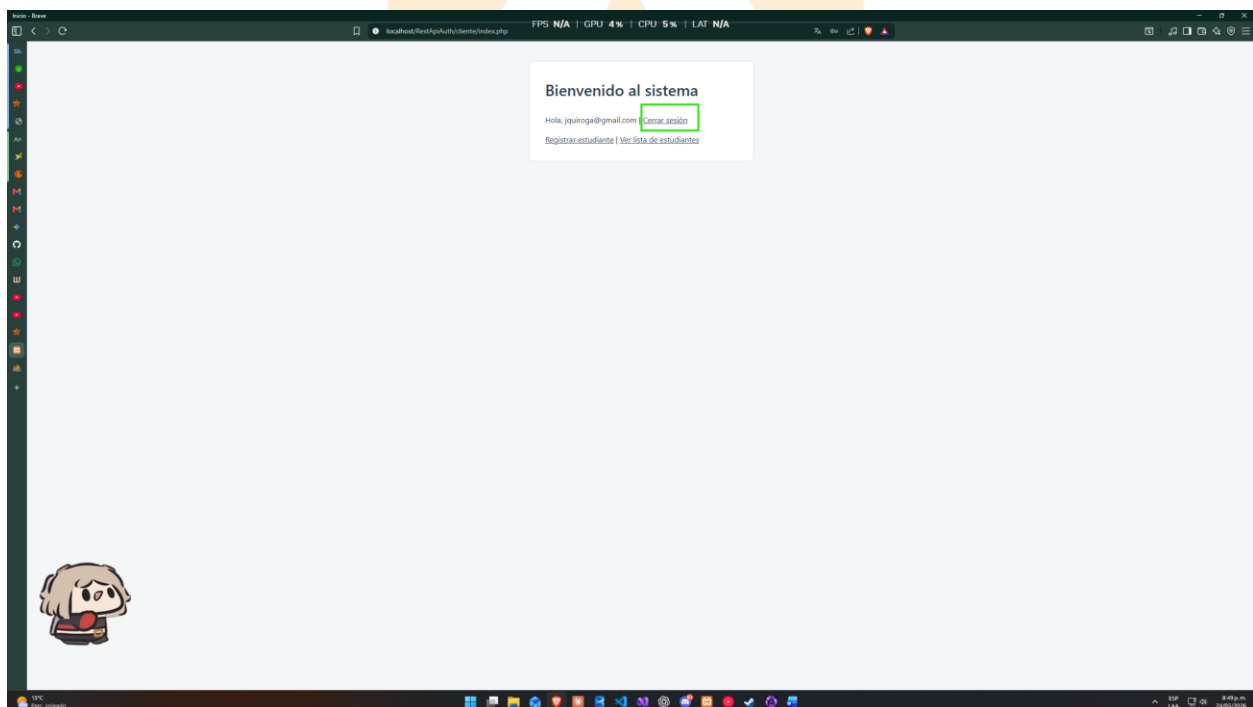
Esta página le dice a la API que cierre la sesión activa y elimine la sesión vigente y lo redirige a la página de inicio de sesión.



The screenshot shows a code editor with a dark theme. The file explorer on the left lists various PHP files, with 'logout.php' selected. The main editor area displays the following PHP code:

```
1 <?php
2 session_start();
3 require_once('Rutas.php');
4
5 if (isset($_SESSION["token"])) {
6     $rutas = new Rutas();
7
8     $ch = curl_init($rutas->getlogoutApiUrl($_SESSION["token"]));
9     curl_setopt($ch, CURLOPT_CUSTOMREQUEST, "DELETE");
10    curl_exec($ch);
11    curl_close($ch);
12 }
13
14 session_destroy();
15 header("Location: index.php");
16 exit;
17 ?>
18
19
```

The bottom status bar of the editor shows system information: FPS N/A, GPU 0%, CPU 7%, LAT N/A.



Gestión de Estudiantes

Este módulo se agregó al proyecto para registrar y consultar estudiantes. Sigue el mismo patrón cliente-servidor que el módulo de autenticación: una clase de acceso a datos, una API, y las páginas del cliente. Permite dos operaciones: crear un estudiante y listar todos los estudiantes.

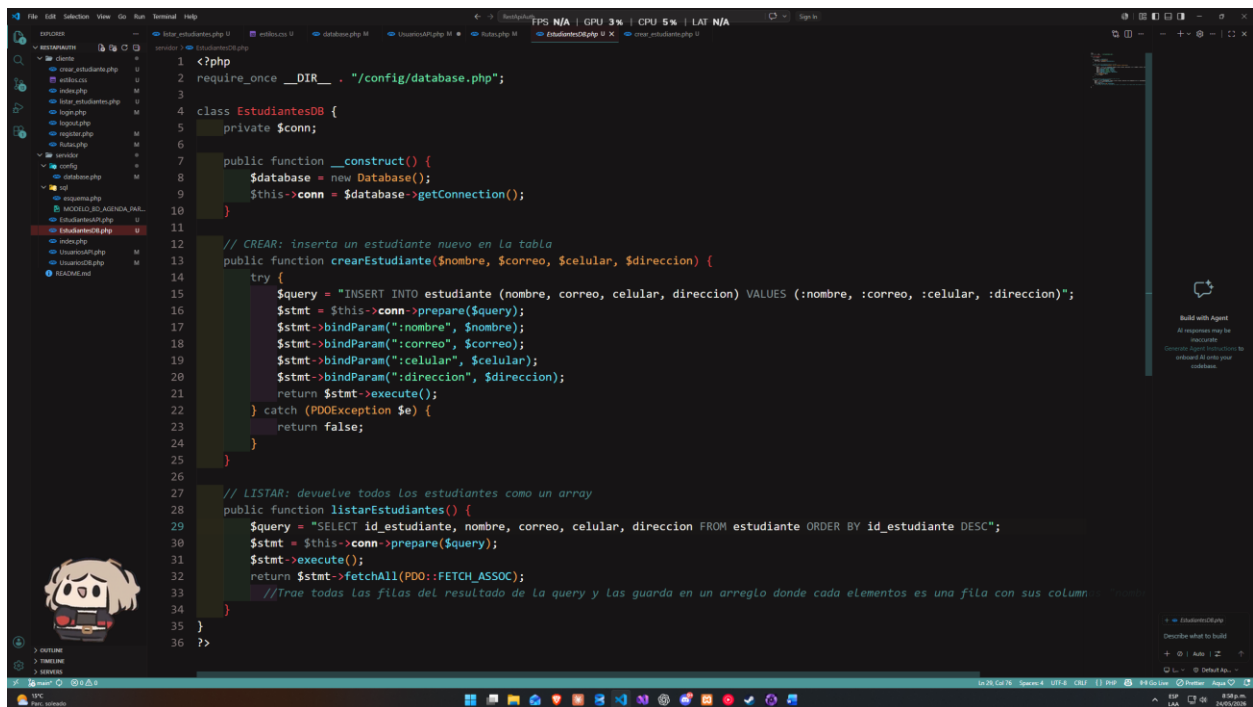
Acceso a los Datos del Estudiante

En esta clase se ejecutan consultas SQL sobre la tabla de estudiante utilizando la conexión definida en **database.php** y contiene dos métodos:

- **crearEstudiante**: realiza una consulta **INSERT** para guardar un nuevo estudiante en la tabla.
- **listarEstudiante**: hace una consulta **SELECT** y devuelve todos los estudiantes que se encuentren registrados.

Para este apartado se hizo uso de **consultas preparadas**. Ambos métodos utilizan consultas preparadas (`prepare` y `bindParam`): los datos del usuario se envían por separado del comando SQL. Esto protege contra vulnerabilidades de seguridad como inyección SQL, un ataque en el que alguien intenta introducir código malicioso a través de los campos de un formulario.

El concepto `fetchAll(PDO::FETCH_ASSOC)` que utiliza el método `listarEstudiantes()` para recoger todas las filas del resultado de una sola vez. `FETCH_ASSOC` indica que cada fila debe entregarse como un arreglo asociativo, donde la clave es el nombre de la columna y el valor es el dato. El resultado es un arreglo de arreglos: una lista en donde cada elemento es un estudiante.



```
1 <?php
2 require_once __DIR__ . "/config/database.php";
3
4 class EstudiantesDB {
5     private $conn;
6
7     public function __construct() {
8         $database = new Database();
9         $this->conn = $database->getConnection();
10    }
11
12    // CREAM: inserta un estudiante nuevo en la tabla
13    public function crearEstudiante($nombre, $correo, $celular, $direccion) {
14        try {
15            $query = "INSERT INTO estudiante (nombre, correo, celular, direccion) VALUES (:nombre, :correo, :celular, :direccion)";
16            $stmt = $this->conn->prepare($query);
17            $stmt->bindParam(":nombre", $nombre);
18            $stmt->bindParam(":correo", $correo);
19            $stmt->bindParam(":celular", $celular);
20            $stmt->bindParam(":direccion", $direccion);
21            return $stmt->execute();
22        } catch (PDOException $e) {
23            return false;
24        }
25    }
26
27    // LISTAR: devuelve todos los estudiantes como un array
28    public function listarEstudiantes() {
29        $query = "SELECT id_estudiante, nombre, correo, celular, direccion FROM estudiante ORDER BY id_estudiante DESC";
30        $stmt = $this->conn->prepare($query);
31        $stmt->execute();
32        return $stmt->fetchAll(PDO::FETCH_ASSOC);
33        //Trae todas las filas del resultado de la query y las guarda en un arreglo donde cada elemento es una fila con sus columnas
34    }
35 }
36 >>
```

API de Estudiantes

Esta clase recibe los métodos HTTP y el parámetro **action** de la petición del cliente para decidir que hacer y llamar al método correspondiente en **EstudiantesDB** y devolver la respuesta en JSON.

Este módulo recibe dos tipos de petición:

- **POST:** para registrar un estudiante. Los datos llegan en el cuerpo de la petición en formato JSON.
- **GET:** para solicitar la lista de los estudiantes que se encuentran registrados.

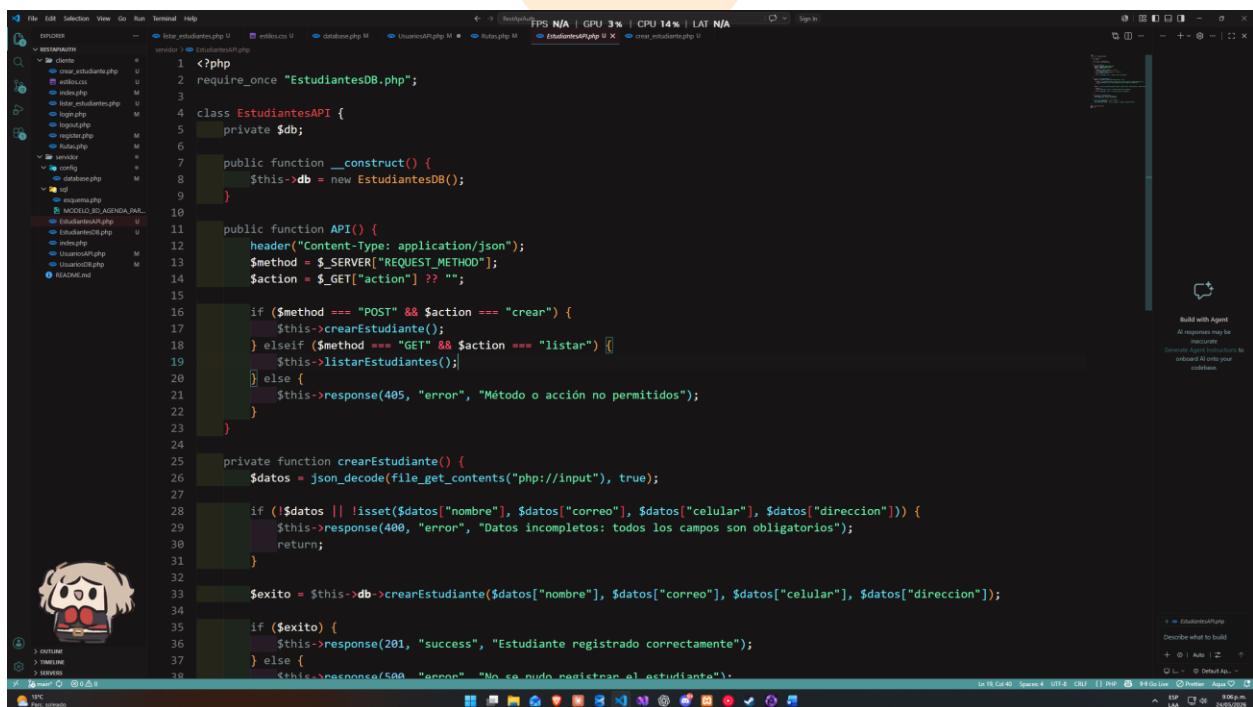
Formulario de Registro de Estudiantes

Esta página contiene el formulario para hacer el registro de estudiantes. Así como en el login este compuesto de dos partes (PHP y HTML) que se ejecutan cuando el cliente envía el formulario.

Aquí se recogen los cuatro campos del formulario, hace una limpieza de caracteres innecesarios o peligrosos, los guarda en un JSON y los envía para el procesamiento de la API.

Listado de Estudiantes

Esta página le solicita a la API la lista de estudiantes mediante una petición GET y la muestra en una tabla. La API recibe un arreglo de estudiantes el cual recorre con un ciclo foreach generando una fila de la tabla por cada estudiante.



```
1 <?php
2 require_once "EstudiantesDB.php";
3
4 class EstudiantesAPI {
5     private $db;
6
7     public function __construct() {
8         $this->db = new EstudiantesDB();
9     }
10
11     public function API() {
12         header("Content-Type: application/json");
13         $method = $_SERVER["REQUEST_METHOD"];
14         $action = $_GET["action"] ?? "";
15
16         if ($method === "POST" && $action === "crear") {
17             $this->crearEstudiante();
18         } elseif ($method === "GET" && $action === "listar") {
19             $this->listarEstudiantes();
20         } else {
21             $this->respuesta(405, "error", "Método o acción no permitidos");
22         }
23     }
24
25     private function crearEstudiante() {
26         $datos = json_decode(file_get_contents("php://input"), true);
27
28         if (!isset($datos["nombre"], $datos["correo"], $datos["celular"], $datos["direccion"])) {
29             $this->respuesta(400, "error", "Datos incompletos: todos los campos son obligatorios");
30             return;
31         }
32
33         $exito = $this->db->crearEstudiante($datos["nombre"], $datos["correo"], $datos["celular"], $datos["direccion"]);
34
35         if ($exito) {
36             $this->respuesta(201, "success", "Estudiante registrado correctamente");
37         } else {
38             $this->respuesta(500, "error", "No se pudo registrar al estudiante");
39         }
40     }
41
42     private function listarEstudiantes() {
43         $estudiantes = $this->db->listarEstudiantes();
44
45         $html = "<table border='1'>";
46         foreach ($estudiantes as $estudiante) {
47             $html .= "<tr>";
48             foreach ($estudiante as $campo => $valor) {
49                 $html .= "<td>$valor</td>";
50             }
51             $html .= "</tr>";
52         }
53         $html .= "</table>";
54
55         $this->respuesta(200, "success", $html);
56     }
57
58     private function respuesta($codigo, $tipo, $mensaje) {
59         $respuesta = [
60             "codigo" => $codigo,
61             "tipo" => $tipo,
62             "mensaje" => $mensaje
63         ];
64         echo json_encode($respuesta);
65     }
66 }
```

```
File Edit Selection View Go Run Terminal Help
FPS N/A | GPU 4% | CPU 7% | LAT N/A

class EstudiantesAPI {
    public function API() {
    }

    private function crearEstudiante() {
        $datos = json_decode(file_get_contents("php://input"), true);

        if (!$datos || !isset($datos["nombre"], $datos["correo"], $datos["celular"], $datos["direccion"])) {
            $this->response(400, "error", "Datos incompletos: todos los campos son obligatorios");
            return;
        }

        $exito = $this->db->crearEstudiante($datos["nombre"], $datos["correo"], $datos["celular"], $datos["direccion"]);

        if ($exito) {
            $this->response(201, "success", "Estudiante registrado correctamente");
        } else {
            $this->response(500, "error", "No se pudo registrar el estudiante");
        }
    }

    private function listarEstudiantes() {
        $estudiantes = $this->db->listarEstudiantes();
        $this->response(200, "success", $estudiantes);
    }

    private function response($code, $status, $message) {
        http_response_code($code);
        echo json_encode(["status" => $status, "message" => $message], JSON_PRETTY_PRINT);
    }
}

$api = new EstudiantesAPI();
$api->API();
?>
```

Registrar Estudiante - Browser

localhost/RestApi/Auth/crear_estudiante.php

FPS N/A | GPU 10% | CPU 10% | LAT N/A

Registrar Estudiante

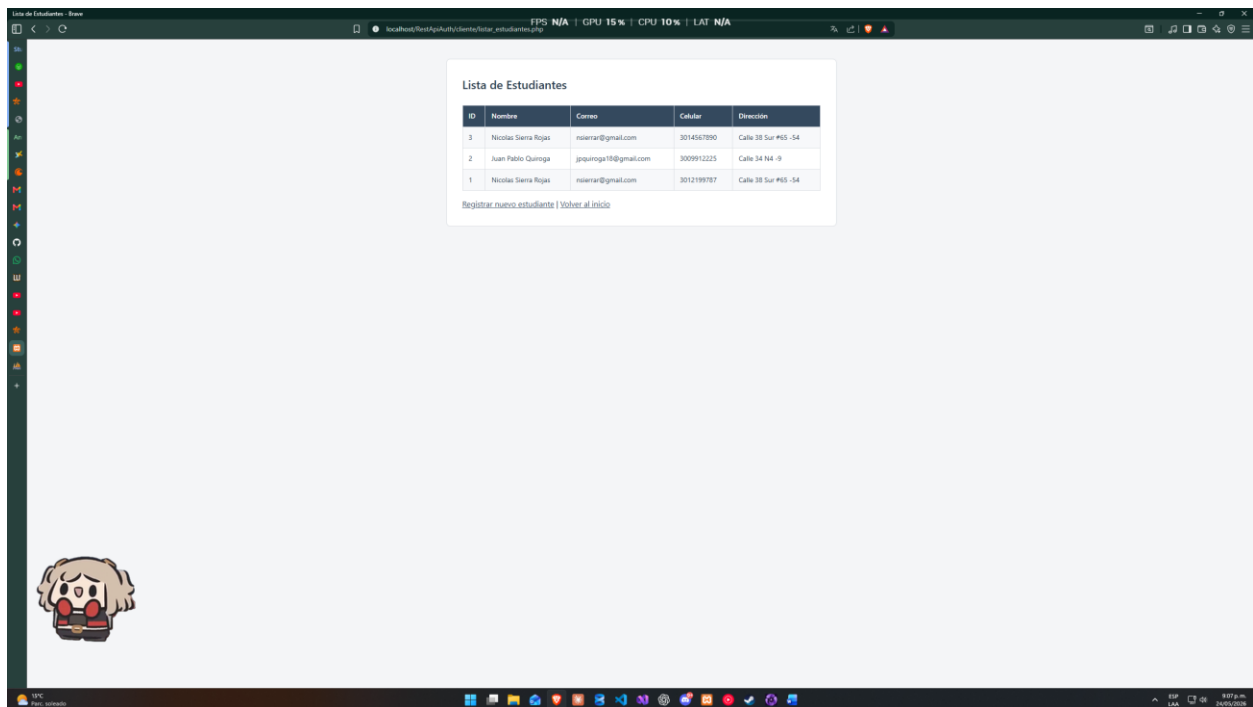
Nombre:

Correo:

Celular:

Dirección:

[Ver lista de estudiantes](#) | [Volver al inicio](#)



Todas las páginas de estas evidencias comparten un archivo de estilos llamada **estilos.css** con el fin de centralizar el estilo de todo el proyecto y simplemente llamarlo desde cada uno de los formularios facilitando cualquier cambio que deseemos realizar en el futuro.

Con esto podemos concluir que el sistema cumple con los objetivos propuestos en la evidencia permitiendo el inicio de sesión con validación de campos y autenticación, el registro de nuevos usuarios y la gestión de estudiantes mediante las operaciones de creación y consulta.